chosen. The problem with this approach is that if the best action (according to the action values) is only slightly better than other options, the worse actions will still be chosen with a fairly high frequency. For example, in the following example we take an action-value tensor for five actions and apply the softmax function from PyTorch's functional module.

```
>>> torch.nn.functional.softmax(th.Tensor([3.6, 4, 3, 2.9, 3.5]))
tensor([0.2251, 0.3358, 0.1235, 0.1118, 0.2037])
```

As you can see, the best action (index 1) is only slightly better than the others, so all the actions have pretty high probability, and this policy is not that much different from a uniformly random policy. We will use a policy that begins with a softmax policy to encourage exploration, and after a fixed number of game steps we will switch to an epsilon-greedy strategy, which will continue to give us some exploration capacity but mostly just takes the best action.

Listing 8.4. The policy function

```
def policy(qvalues, eps=None):
1
    if eps is not None:
        if torch.rand(1) < eps:
            return torch.randint(low=0,high=7,size=(1,))
        else:
            return torch.argmax(qvalues)
    else:
        return torch.multinomial(F.softmax(F.normalize(qvalues)),
num_samples=1)2
```

- ***1*** The policy function takes a vector of action values and an epsilon (eps) parameter.
- ***2*** If eps is not provided, uses a softmax policy. We sample from the softmax using the multinomial function.

The other big component we need for the DQN is an *experience replay memory*. Gradient-based optimization does not work well if you only pass one sample of data at a time because the gradients are too noisy. In order to average over the noisy gradients, we need to take sufficiently large samples (called batches or mini-batches) and average or sum the gradients over all the samples. Since we only see one sample of data at a time when playing a